

Project Testing – EduGenie-AI

Google Gemini Powered AI Learning Assistant

Page 1 – Testing Overview and Objectives

The Project Testing phase verifies whether EduGenie-AI works according to the requirements and design. Testing checks the frontend, FastAPI backend, Gemini integration, learning modules, error handling, and complete user workflow.

Testing Objectives

- Verify that all major application features work correctly.
- Check that frontend requests reach the correct FastAPI endpoints.
- Verify successful communication with Google Gemini.
- Identify errors in question answering, explanations, quizzes, summaries, and recommendations.
- Check application behavior for invalid or empty inputs.
- Confirm that sensitive API credentials are not exposed.
- Ensure the complete student workflow works from input to displayed result.

Testing Approach

- Unit-level testing of individual learning modules.
- Integration testing between frontend, backend, modules, and Gemini.
- Functional testing of every major feature.
- Negative testing using invalid, empty, or unexpected inputs.
- Usability testing of the student interface.
- Security checks for environment variables and source-control configuration.

Testing Environment

Item	Testing Setup
Operating Environment	Windows development environment
Backend	FastAPI with Uvicorn
AI Service	Google Gemini
Frontend	HTML, CSS and JavaScript
Browser	Modern web browser
Development Tool	Visual Studio Code
Version Control	Git and GitHub

Page 2 – Functional and Module Testing

Functional testing checks whether each EduGenie-AI feature produces the expected type of result when valid input is provided.

Test Cases for Core Features

Test ID	Feature	Test Input	Expected Result
TC01	Home Page	Open application URL	EduGenie-AI page loads
TC02	Health Check	GET /health	Status response is returned
TC03	Q&A	College subject question	Clear AI-generated answer
TC04	Explanation	Topic name	Step-by-step explanation
TC05	Quiz	Subject/topic	Quiz content is generated
TC06	Summary	Learning text	Concise summary is returned
TC07	Recommendations	Learning topic	Learning recommendations are returned

Module Testing Process

- Run each module with a normal valid input.
- Check whether the module sends the correct task to Gemini.
- Verify that a meaningful response is returned.
- Check that the response reaches the frontend correctly.
- Repeat with different topics to confirm general behavior.
- Record any unexpected output or error for debugging.

Expected Functional Behavior

A successful functional test should show that the requested EduGenie-AI feature responds without application crashes and returns content relevant to the supplied input. The result should be displayed clearly in the web interface.

Page 3 – Integration, Negative and Error Testing

Integration testing verifies communication between the different parts of EduGenie-AI. Negative testing checks how the application behaves when the input or service conditions are not ideal.

Integration Test Cases

Test ID	Integration Area	Test	Expected Result
IT01	Frontend → FastAPI	Submit a Q&A request	Backend receives request
IT02	FastAPI → Module	Call explanation route	Correct module processes input
IT03	Module → Gemini	Generate response	Gemini returns content
IT04	Gemini → Backend	Return generated text	Backend receives response
IT05	Backend → Frontend	Display result	Result appears in UI

Negative and Error Test Cases

Test	Input / Condition	Expected Handling
Empty input	No question or topic	Show validation message or prevent request
Very long input	Large text	Process safely or provide useful limitation message
Invalid endpoint	Wrong URL	Return appropriate HTTP error
Gemini failure	AI service unavailable	Return understandable error
Invalid API configuration	Missing/incorrect key	Application reports configuration problem
Unexpected response	AI output differs from expected format	Handle without crashing
Network interruption	Connection unavailable	Show failure message

Error Handling Verification

Testing should confirm that errors do not expose secret credentials or unnecessary internal information. The application should remain stable and provide a useful message that helps the user understand that the request could not be completed.

Page 4 – Usability, Security and Performance Testing

Testing is not limited to functionality. EduGenie-AI should also be easy to use, reasonably responsive, and configured safely.

Usability Testing

- Check whether feature labels are easy to understand.
- Verify that input fields are clearly visible.
- Check whether results are readable and properly displayed.
- Confirm that users can understand what to do without technical knowledge.
- Check error messages for clarity.
- Test the interface using different common learning questions.

Security Testing

- Verify that GEMINI_API_KEY is stored in .env.
- Confirm .env is included in .gitignore.
- Check that the API key is not present in frontend files.
- Avoid displaying credentials in terminal or application errors.
- Review Git status before committing project files.
- Verify that only required project files are shared in the repository.

Performance and Reliability Checks

- Check how quickly normal requests receive responses.
- Verify that the server remains available during repeated requests.
- Test several features one after another.
- Observe behavior when Gemini temporarily cannot respond.
- Ensure the application does not freeze because of an unsuccessful request.

Testing Tools and Methods

Tool / Method	Purpose
Browser	Test user interface and displayed results
FastAPI endpoints	Verify backend request handling
Terminal	Observe server logs and errors
Direct Gemini test	Verify AI API connectivity independently
Git status	Check that secret files are not staged
Manual test cases	Verify expected feature behavior

Page 5 – Test Results, Defect Handling and Conclusion

The final testing stage records results, identifies defects, applies fixes, and repeats the affected tests. A feature should be marked complete only after its important test cases pass.

Sample Test Result Format

Test ID	Expected Result	Actual Result	Status
TC01	Home page loads	Page loads	Pass
TC02	Health response returned	Status returned	Pass
TC03	Q&A answer generated	Answer displayed	Pass
TC04	Explanation generated	Explanation displayed	Pass
TC05	Quiz generated	Quiz content displayed	Pass
TC06	Summary generated	Summary displayed	Pass
TC07	Recommendations generated	Recommendations displayed	Pass

Defect Management Process

- Identify the failing test case.
- Record the error message and affected component.
- Determine whether the problem is frontend, backend, module, configuration, or Gemini related.
- Apply the smallest appropriate fix.
- Run the failed test again.
- Run related tests to ensure the fix did not break another feature.
- Update the test result and document important defects.

Final Testing Checklist

- All major FastAPI routes tested.
- Gemini connection verified.
- Learning modules tested with valid inputs.
- Empty and invalid inputs checked.
- Frontend-backend integration verified.
- Security configuration reviewed.
- Important errors handled without application crashes.
- Final demonstration workflow tested.

Conclusion

The Project Testing phase ensures that EduGenie-AI is reliable, usable, and aligned with its requirements. Functional, integration, negative, usability, security, and basic performance checks provide confidence that the application can be demonstrated and documented as a complete academic project.